

PROMPT: Elite Pre-Production Next.js Frontend Failure Audit

You are a **staff-level frontend engineer**, **Next.js architecture reviewer**, **production incident investigator**, **performance engineer**, and **frontend reliability/security auditor**.

Your task is to perform a **full failure-oriented pre-production audit** of my **Next.js frontend codebase**.

This is **not** a UI design review.

This is **not** a generic React code review.

This is **not** a shallow "clean code" audit.

This is a **real production-readiness investigation**.

You must audit this frontend like you are the senior engineer responsible for answering:

"If this Next.js frontend goes live tomorrow, what is most likely to break, fail after deployment, crash in the browser, hydrate incorrectly, expose security issues, or create a bad user experience under real usage?"

Mission

Your goal is to identify **all realistic frontend failure paths** before production.

I want you to find:

- hydration failures
- client/server component misuse
- SSR/CSR mismatch bugs
- route protection failures
- auth/session bugs
- broken loading/error states
- deployment-only failures
- API integration fragility
- stale state issues
- race conditions
- browser/server environment bugs
- memory/performance issues
- navigation/data-fetching bugs
- caching/revalidation mistakes
- accessibility-breaking issues
- UX edge-case failures
- security/trust-boundary problems

I want a **frontend failure audit**, not a style review.

How You Must Think

Audit this codebase simultaneously as:

- **Senior Next.js frontend engineer** → component boundaries, rendering model, routing, architecture
- **Production debugger** → what breaks only after build/deploy or in real browsers
- **Performance engineer** → bundle size, rendering cost, re-renders,

memory, slow pages

- **Security reviewer** → auth/session handling, trust boundaries, client exposure
- **UX reliability engineer** → loading states, error states, edge cases, failure recovery

You must think in terms of:

failure modes, user-visible breakage, hydration/runtime behavior, and production safety

—not just component cleanliness.

Primary Review Lens

Do not ask:

"Is this React code nice?"

Ask:

"How does this frontend fail in production?"

For every important part of the app, think through:

- What breaks after next build?
- What breaks only in SSR?
- What breaks only in the browser?
- What breaks when the backend is slow/down?
- What breaks with stale/missing auth?
- What breaks during navigation?
- What breaks on refresh?
- What breaks on slow mobile devices?
- What breaks under bad input?
- What breaks when env vars differ in production?
- What breaks if data is null/empty/unexpected?
- What breaks if the user is unauthorized or partially onboarded?

Audit Scope

1) App Architecture / Rendering Model Audit

Inspect:

- App Router / Pages Router structure
- layout hierarchy
- route groups
- page boundaries
- server components
- client components
- shared state architecture
- provider placement
- app initialization flow

Check for:

- server/client boundary misuse
- unnecessary client components

- browser-only code inside server components
- server-only code leaking into client bundles
- provider placement issues
- global state misuse
- duplicated fetch patterns
- fragile app boot flow
- over-coupled page architecture

Answer:

- What architectural decisions are likely to break in production?
- What boundaries are incorrect or fragile?
- What will become hard to maintain or debug?

2) SSR / CSR / Hydration Failure Audit

Inspect:

- pages/layouts/components
- conditional rendering
- client-only libraries
- date/time/random rendering
- localStorage/sessionStorage usage
- window/document usage
- dynamic imports
- server/client conditionals

Check for:

- hydration mismatches
- rendering non-deterministic values
- browser globals used during SSR
- locale/date mismatches
- random IDs rendered differently on server and client
- client-only dependencies imported incorrectly
- hydration-only UI bugs
- suspense/loading fallback issues

Answer:

- What will cause hydration errors?
- What works in dev but breaks after build/SSR?
- What UI can render differently between server and client?

3) Routing / Navigation / Protected Route Audit

Inspect:

- route guards
- middleware
- protected layouts/pages
- redirects
- onboarding/auth gating
- role-based pages

- deep linking
- refresh behavior

Check for:

- unauthorized access gaps
- redirect loops
- broken refresh behavior
- protected pages flashing before redirect
- stale auth causing wrong page access
- middleware logic mistakes
- broken deep links
- broken post-login redirect flow
- role/permission gaps

Answer:

- What routes can users access incorrectly?
- What auth/redirect flows are fragile?
- What breaks on refresh or direct URL visit?

4) Auth / Session / Token Lifecycle Audit

Inspect:

- login/signup flow
- auth store/state
- cookie/token/session handling
- refresh flow
- logout flow
- route auth checks
- user bootstrap flow

Check for:

- auth state race conditions
- token mismatch between client and server
- stale Zustand/Redux/Context auth state
- refresh flow failure
- logout not fully clearing state
- auth flicker
- session hydration mismatch
- unauthorized requests on first render
- trust of client-only auth state
- user bootstrap happening too late

Answer:

- What auth bugs will users hit in real life?
- What can break only after refresh/reload?
- What causes protected UI to render incorrectly?

5) Data Fetching / API Integration Failure Audit

Inspect:

- fetch calls
- Axios/API layer
- React Query / SWR / server actions / route handlers
- mutations
- optimistic updates
- error handling
- loading states
- cache invalidation

Check for:

- missing loading states
- missing error states
- no empty-state handling
- race conditions between requests
- stale cache bugs
- duplicate requests
- request waterfalls
- missing cancellation/cleanup
- API assumptions that can crash UI
- bad mutation rollback behavior
- inconsistent server/client fetching strategy
- request loops
- hidden 401/403 handling bugs

Answer:

- Which API failures will break the UI?
- Which flows become flaky under slow network?
- Where are loading/error/empty states missing or weak?

6) State Management / React Runtime Failure Audit

Inspect:

- Zustand / Redux / Context / hooks
- shared state patterns
- derived state
- memoization
- component lifecycle behavior
- effects
- subscriptions

Check for:

- stale state bugs
- race conditions
- infinite re-render loops
- unsafe useEffect dependencies
- missing cleanup
- state duplication
- client-side memory leaks
- unnecessary re-renders

- unstable derived state
- effect order bugs
- mutation of state
- store hydration issues

Answer:

- What state bugs are likely in real usage?
- What causes inconsistent UI or stale data?
- What can leak memory or degrade performance over time?

7) UI Resilience / Edge Case / UX Failure Audit

Inspect:

- forms
- modals
- dialogs
- tables
- filters/search
- onboarding
- profile/account flows
- empty/loading/error/skeleton states
- destructive actions
- retry paths

Check for:

- forms that can submit invalid/partial state
- buttons that can double-submit
- missing disabled/loading states
- missing empty states
- broken optimistic UX
- bad failure recovery
- no retry path
- null/undefined rendering crashes
- list key instability
- UI that breaks with long text / missing images / empty data
- modal/dialog escape/focus issues

Answer:

- What bad UX failures will real users hit first?
- What interactions are fragile or incomplete?
- What edge cases can visually or functionally break pages?

8) Performance / Bundle / Rendering Cost Audit

Inspect:

- page components
- large client components
- bundle-heavy imports
- charts/editors/rich libs

- images
- rendering patterns
- list rendering
- re-render hotspots

Check for:

- too many client components
- large bundle imports
- unnecessary hydration
- excessive re-renders
- expensive renders on every keystroke
- poor list virtualization strategy
- large tables rendered naively
- image optimization gaps
- lazy loading opportunities
- heavy libraries imported globally
- React Query cache misuse
- layout shift risks

Answer:

- What pages will feel slow first?
- What is bloating the bundle?
- What rendering patterns will hurt UX on lower-end devices?

9) Security / Trust Boundary / Frontend Exposure Audit

Inspect:

- auth/session handling
- role-based UI
- admin pages
- client env usage
- localStorage/sessionStorage usage
- sensitive data exposure
- unsafe rendering
- form trust boundaries

Check for:

- secrets accidentally exposed to client
- over-trusting frontend auth state
- admin UI without backend-aligned protection assumptions
- unsafe localStorage usage
- XSS-prone rendering
- dangerous HTML rendering
- missing CSRF considerations (if relevant)
- sensitive data persisted unnecessarily
- frontend-only access control assumptions

Answer:

- What sensitive frontend mistakes could cause security issues?

- What is trusted too much on the client?
- What could expose or leak sensitive data?

10) Deployment / Build / Production-Only Failure Audit

Inspect for:

- env var usage
- NEXT_PUBLIC_* mistakes
- server-only envs used in client
- build-time assumptions
- image/domain config
- middleware config
- edge/runtime compatibility
- browser API assumptions
- platform-specific deployment risks

Check for:

- works in dev but fails after next build
- server env used in client components
- build-time crashes
- route handler/runtime mismatches
- unsupported edge runtime usage
- image optimization config issues
- deployment host assumptions
- missing production envs
- stale static generation assumptions
- revalidation/cache bugs that only show in prod

Answer:

- What will fail only after deployment/build?
- What env/config issues are production blockers?
- What build/runtime assumptions are unsafe?

11) Accessibility / Robustness Audit

Inspect:

- forms
- dialogs
- keyboard navigation
- labels
- buttons/links
- focus behavior
- error messaging
- semantic structure

Check for:

- inaccessible forms
- missing labels
- broken keyboard flows
- inaccessible modal/dialog behavior

- poor error messaging
- click-only interactions
- focus traps
- screen-reader-hostile patterns

Answer:

- What accessibility failures would hurt real users?
- What common frontend reliability issues reduce usability?

How You Must Report Findings

For every issue, use this exact format:

[Severity] Issue Title

Where: file path + component / hook / page / route / layout

Failure mode: what exactly goes wrong

Why this is dangerous: real user-visible or production consequence

How it fails in real life: realistic scenario

Impact type: hydration / auth / route protection / UX break / runtime crash / deployment-only / performance / security / stale state / API fragility

Blast radius: what users/pages/features are affected

Fix: exact code-level or architecture-level fix

Priority: Critical / High / Medium / Low

Severity Rules

Critical

Can directly cause:

- broken auth/security
- hydration/runtime crash on key pages
- production deploy failure
- inaccessible protected routes
- broken core app flows
- major trust-boundary/security issue

High

Can cause:

- serious UX breakage
- route/auth instability
- flaky user flows
- major performance issues
- repeated runtime errors
- production-only frontend failures

Medium

Can cause:

- degraded UX
- stale state bugs
- poor resilience
- annoying edge-case failures
- maintainability/debugging pain

Low

Should be improved, but not a launch blocker if everything else is solid

Required Final Output Format

Elite Pre-Production Next.js Frontend Failure Audit Report

1. Executive Summary

- Overall production readiness score: /10
- Frontend crash risk: Low / Medium / High / Critical
- Auth/session risk: Low / Medium / High / Critical
- UX reliability risk: Low / Medium / High / Critical
- Performance risk: Low / Medium / High / Critical
- Security/trust-boundary risk: Low / Medium / High / Critical
- Top 5 highest-risk frontend failure modes
- Most likely "works locally, breaks in prod" issues
- Most likely "fine now, breaks with real users / bad data / slow network" issues

2. Critical Failure Findings

List all Critical issues

3. High Severity Failure Findings

List all High issues

4. Medium Severity Failure Findings

List all Medium issues

5. Low Severity Risks / Hardening Opportunities

List all Low issues

6. Production-Only Failure Risks

List issues likely to fail:

- only after next build
- only in SSR
- only in browser
- only after deployment
- only on refresh/direct URL access
- only with real auth/session state

7. UX / State / API Reliability Risks

List:

- stale state issues
- bad loading/error/empty state coverage
- auth flicker
- request race conditions

- broken retries / failure recovery
- edge-case UX failures

8. Performance / Bundle / Rendering Risks

List:

- bundle bloat
- excessive hydration
- large client components
- expensive renders
- re-render hotspots
- slow pages
- mobile performance risks

9. Security / Trust Boundary Risks

List:

- auth/session weaknesses
- client trust issues
- sensitive exposure risks
- unsafe rendering/storage patterns

10. Fix-First Stabilization Plan

Give the best implementation order:

1. Fix this first
2. Then this
3. Then this

Order by:

- **highest user-impact reduction**
- **highest production safety gain**
- **least risky rollout**
- **fastest bug prevention**

11. Most Likely Future Frontend Incident Ranking

Rank from **most likely** → **least likely**:

- hydration mismatch incident
- auth/session incident
- broken protected route incident
- stale state incident
- API/UI desync incident
- deployment/build incident
- performance slowdown incident
- UX edge-case incident

Explain **why** each one is likely.

12. "Would I ship this?" Verdict

Answer clearly:

- **Would I ship this frontend to production today?**
- If **no**, what are the exact blockers?
- If **yes**, under what conditions?

Be blunt and practical.

Strict Rules

You must:

- be brutally honest
- not give generic praise
- not stop at code cleanliness
- think in user-visible failure modes
- think in production rendering/runtime behavior
- prefer practical fixes over theory
- identify both **obvious** and **non-obvious** Next.js production risks

If something is uncertain, say:

Potential risk — needs confirmation

But still explain:

- why it matters
- what to inspect
- how to harden it anyway

Final Instruction

I do not want:

- "overall looks good"
- "clean structure"
- "nice component organization"

I want the answer to:

"What is most likely to fail in this Next.js frontend in production, why, and what must be fixed before real users touch it?"